University of BRISTOL

Architecture II

Guosheng Hu

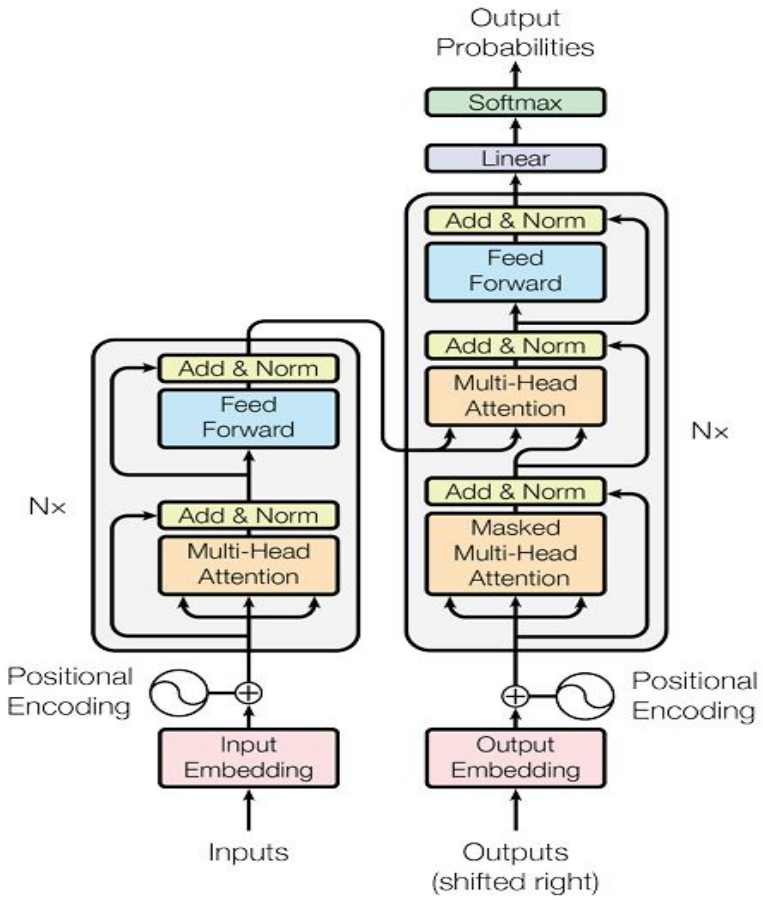
Motivation

The animal didn’t cross the street because it was tired.

What does “it” refer to?

Self-attention lets each word decide which other words matter most.
Each word asks: who should I pay attention to in this sentence?

Self-Attention vs. Transformer



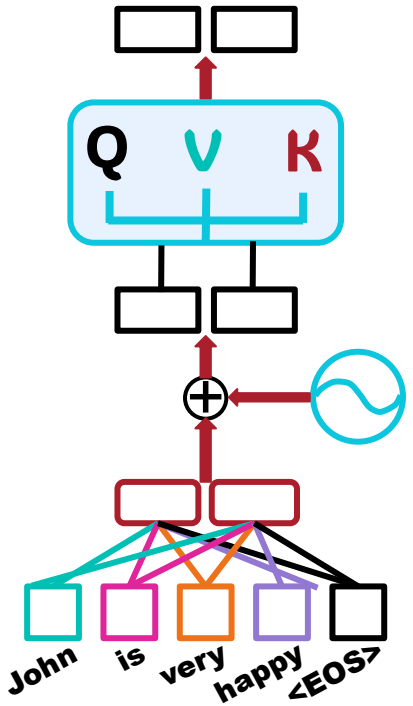
Self-attention is the core operation/mechanism.

The Transformer is an architecture built almost entirely out of self-attention.

Vaswani, Ashish, et al. "Attention is all you need." *NIPS* (2017).

Transformer

3 main parts: Attention, Positional Encoding, Word Embedding



Attention — helps establish relationships among words

Positional Encoding — helps keep tracks of word order

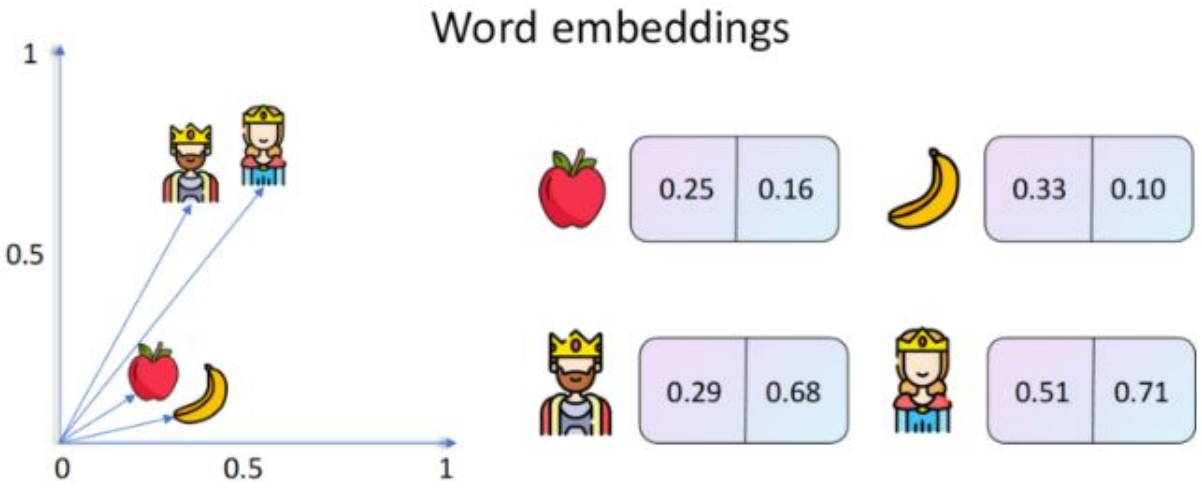
Word Embedding — converts the input words into numbers

Word Embedding

A **word embedding** represents a word as a **vector of numbers** that captures its meaning.

"People" → [0.20, -0.70, 0.88, ...]
"are" → [0.05, -0.10, 0.02, ...]
"laughing" → [0.60, 0.20, -0.30, ...]

Similar meanings → similar vectors



One Embedding Function learns the projection from word to embedding.

Each word → unique index in vocabulary→ one-hot vector → input to embedding function

Positional Encoding - Motivation

Self-Attention Doesn’t Know Word Order. Self-attention treats a sentence as a bag of words unless we explicitly add position information.

For Example,
Sentence A: dog bites man
Sentence B: man bites dog

Positional Encoding - How

Each word has an **embedding** (meaning)

Each position has a **positional encoding** (order)

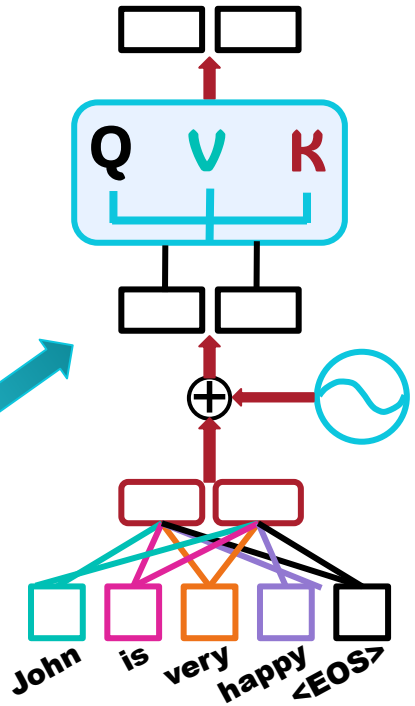
We **add them together**

"people" → [0.2, 0.7, ...]

+

pos → [0.1, 0.0, ...]

= [0.3, 0.7, ….]



Positional Encoding - How

Nearby positions → similar vectors, Far positions → different vectors

‘People are laughing’

Word	Position	Positional Encoding
People	1	[0.10, 0.99]
are	2	[0.20, 0.98]
laughing	3	[0.30, 0.95]

Fixed

- Uses sine & cosine waves
- No learning
- Can generalise to longer sequences

Learned

- Position vectors are learned like embeddings
- Flexible, often used in practice

Self-Attention

Equation for calculating Self-Attention:

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q: Query **K: Key** **V: Value**

Query, **Key**, and **Value** come from database terminology

Self-Attention - Query, Key, Value

Understanding from database terminology

A student database that **paired** each students' **Last Name** with their **Score.**

Student Database

Last Name	Score
John	94
Jonny	62
Jenny	75
...	...

Search for **'John'**, but type **'Jehn'** by mistake.

Computer has to find the **Last Name** in the dataset closest to **'Jehn'**.

Self-Attention - Query, Key, Value

Understanding from database terminology

Student Database

Last Name	Exam Score
John	75
Jonny	84
Jenny	61
…	…

Rank
1st
3rd
2nd
…

A student database that paired each students' **Last Name** with their **Exam Score.**

In database terminology:

- The search term **'Jehn'** called a **Query.**
- The actual names in the database called the **Keys.**

Calculates similarities between the **Query** and all of the **Keys** in the database and ranks each ones:

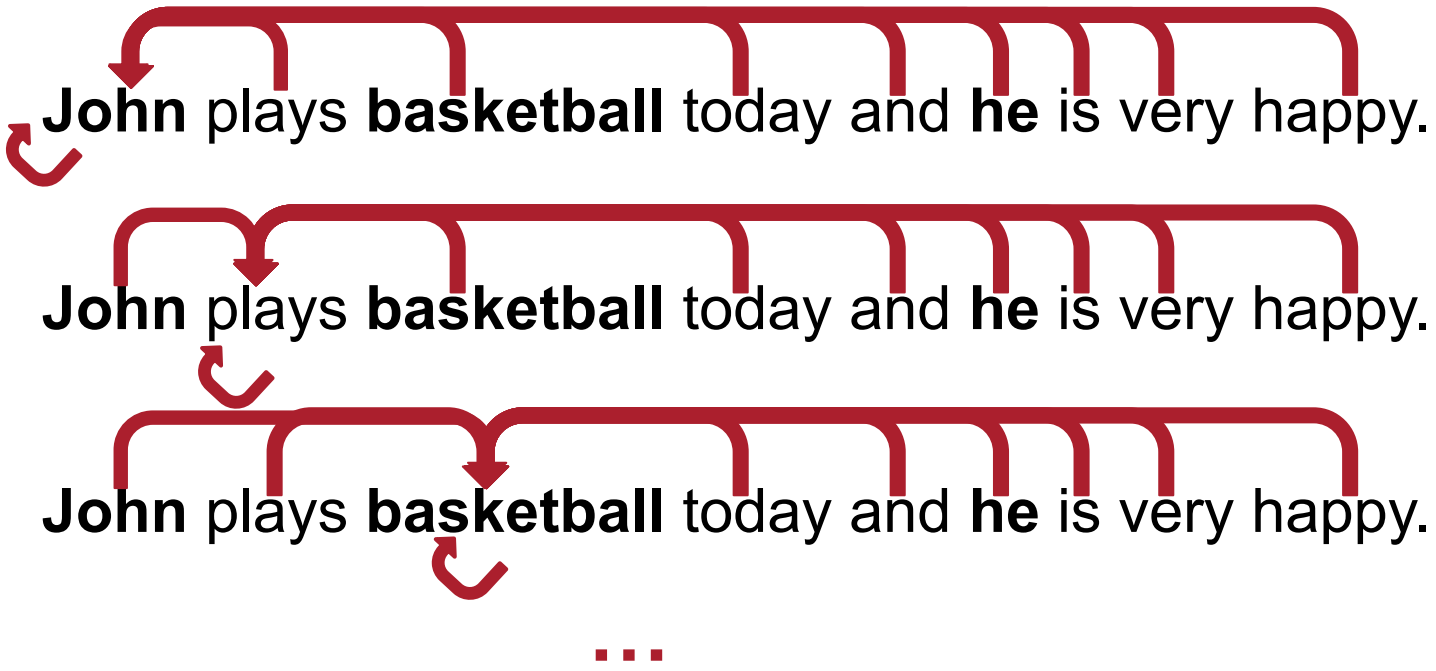
the **Query** **'Jehn'** is closest to the **Key** **'John'**, and return the corresponding exam score **'75'**.

- The returned exam score **'75'** called the **Value**.

Self-Attention - Query, Key, Value

Understanding in the context of a Transformer

Self-Attention calculates similarities between each word and itself and all of the other words.



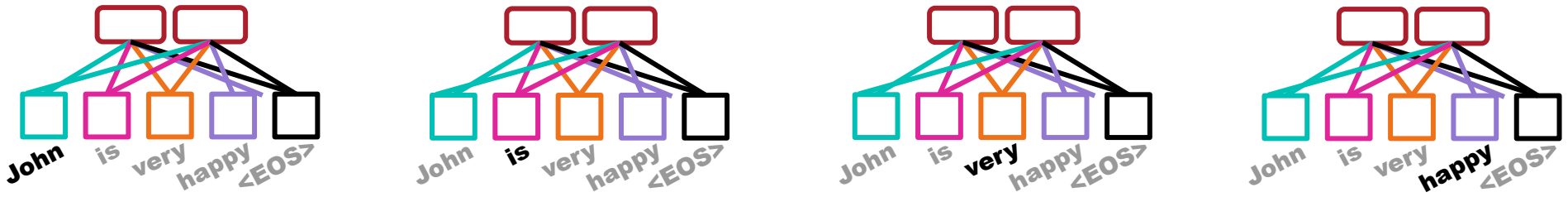
We calculate a **Query** and a **Key** for each word and each Key needs to return a **Value**

Self-Attention

Understanding in the context of a Transformer

Example with a more simple prompt: **John is very happy**

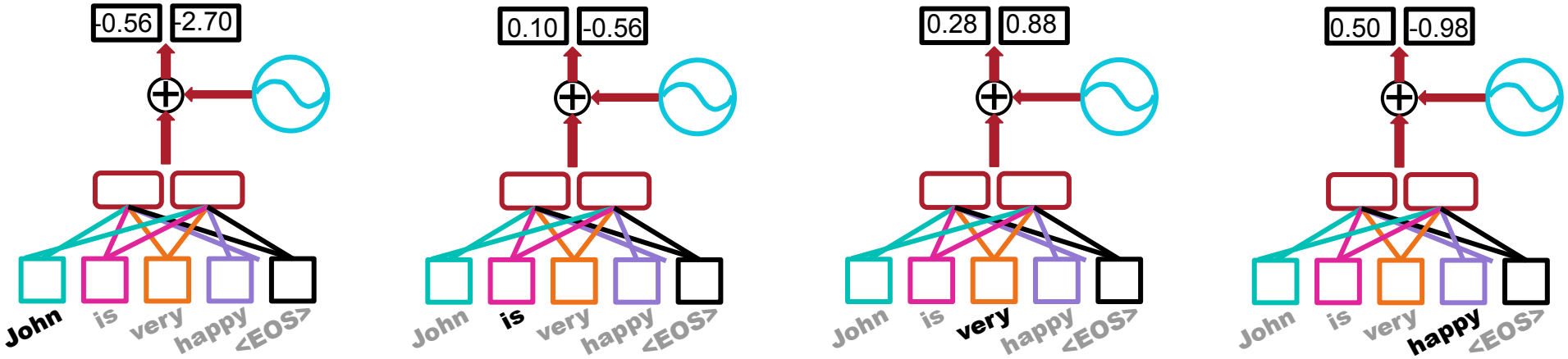
First, convert each word in the prompt into **Word Embedding.**



Self-Attention

Example with a more simple prompt: **John is very happy**

Second, add **Positional Encoding** to the **Word Embedding.**



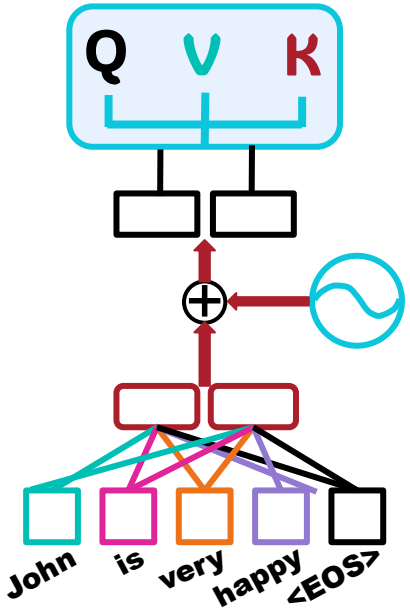
Encoded Values:

John	**-0.56**	**-2.70**
is	**0.10**	**-0.56**
very	**0.28**	**0.88**
happy	**0.50**	**-0.98**

NOTE: In this example, only **2 number** is used to represent each word, instead of **512 or more** in practice.

Self-Attention

Calculating the Q, K and V

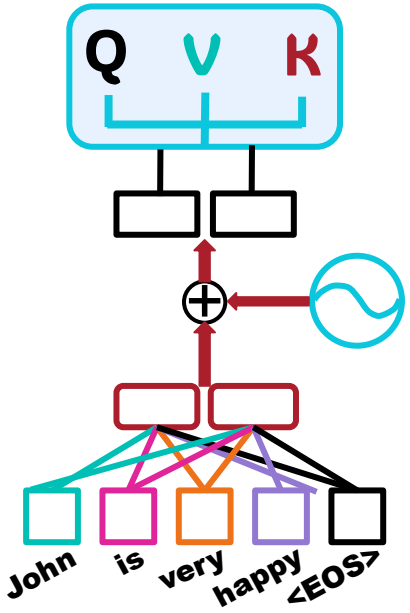


	Encoded Values			Query Weights^T			Q		
John	-0.56	-2.70	✖	-1.63	-0.06	=	1.79	-1.94	John
is	0.10	-0.56		-0.32	0.73		0.02	-0.41	is
very	0.28	0.88					-0.74	0.62	very
happy	0.50	-0.98					-0.51	-0.75	happy

Multiply the **Encoded Values** by the matrix of **Query Weights** (*learnable linear projection*) to calculate 2 **Query** numbers per word.

Self-Attention

Calculating the Q, K and V

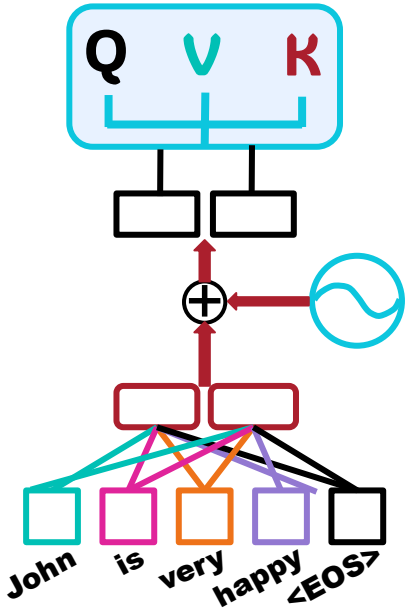


	Encoded Values			Key Weights^T			K		
John	-0.56	-2.70	✖	-1.13	-0.81	=	2.59	-2.36	John
is	0.10	-0.56		-0.72	1.04		0.29	-0.66	is
very	0.28	0.88					-0.95	0.68	very
happy	0.50	-0.98					0.14	-1.43	happy

Multiply the **Encoded Values** by the matrix of **Key Weights** (*learnable linear projection*) to calculate 2 **Key** numbers per word

Self-Attention

Calculating the Q, K and V



	Encoded Values			Value Weights^T			V		
John	-0.56	-2.70	✖	0.61	-0.17	=	-5.21	4.69	John
is	0.10	-0.56		1.8	-1.7		-0.94	0.93	is
very	0.28	0.88					1.75	-1.54	very
happy	0.50	-0.98					-1.46	1.58	happy

Multiply the **Encoded Values** by the matrix of **Value Weights** (*learnable linear projection*) to calculate 2 **Value** numbers per word

Self-Attention

Following the equation:

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multiply the Query matrix by the transpose of the Key matrix.

Q

John	1.79	-1.94
is	0.02	-0.41
very	-0.74	0.62
happy	-0.51	-0.75

✖

K^T

2.59	0.29	-0.95	0.14
-2.36	-0.66	0.68	-1.43

=

Unscaled Dot Product Similarities

	John	is	very	happy
John	9.21	1.80	-3.02	3.02
is	1.02	0.28	-0.30	0.59
very	-3.38	-0.62	1.12	-0.99
happy	0.45	0.35	-0.03	1.00

Self-Attention

Following the equation:

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scale the matrix of dot product similarities.

d_k is the dimension of the Key matrix, which is 2, not the number of token (4).

Unscaled Dot Product Similarities

	John	is	very	happy
John	9.21	1.80	-3.02	3.02
is	1.02	0.28	-0.30	0.59
very	-3.38	-0.62	1.12	-0.99
happy	0.45	0.35	-0.03	1.00

$$\frac{\quad}{\sqrt{2}}$$

=

Scaled Dot Product Similarities

	John	is	very	happy
John	6.52	1.27	-2.14	2.14
is	0.72	0.20	-0.21	0.42
very	-2.39	-0.44	0.80	-0.70
happy	0.32	0.25	-0.02	0.71

Self-Attention

Following the equation:

attention weight

attention score

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Take the SoftMax of each row in the matrix of scaled Dot Product similarities.

Scaled Dot Product Similarities

	John	is	very	happy	
SoftMax(	6.52	1.27	-2.14	2.14	) John
SoftMax(	0.72	0.20	-0.21	0.42	) is
SoftMax(	-2.39	-0.44	0.80	-0.70	) very
SoftMax(	0.32	0.25	-0.02	0.71	) happy

=

SoftMax(**Scaled Dot Product Similarities**)

	John	is	very	happy	
	0.98	0.01	0.00	0.01	John
	0.37	0.22	0.14	0.28	is
	0.03	0.19	0.64	0.14	very
	0.24	0.23	0.17	0.36	happy

Self-Attention

Following the equation:

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multiply by the Values in matrix V.

SoftMax(**Scaled Dot Product Similarities**)

	John	is	very	happy
John	0.98	0.01	0.00	0.01
is	0.37	0.22	0.14	0.28
very	0.03	0.19	0.64	0.14
happy	0.24	0.23	0.17	0.36

✖

V

-5.21	4.69
-0.94	0.93
1.75	-1.54
-1.46	1.58

=

Self-Attention values

-5.14	4.63
-2.25	2.12
0.61	-0.47
-1.69	1.65

Self-Attention

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

SoftMax(**Scaled Dot Product Similarities**)

	John	is	very	happy
John	0.98	0.01	0.00	0.01
is	0.37	0.22	0.14	0.28
very	0.03	0.19	0.64	0.14
happy	0.24	0.23	0.17	0.36

✖

V

-5.21	4.69
-0.94	0.93
1.75	-1.54
-1.46	1.58

=

Self-Attention values

-5.14	4.63
-2.25	2.12
0.61	-0.47
-1.69	1.65

$$(0.983, 0.005, 0, 0.012) \times \begin{pmatrix} -5.21 & 4.69 \\ -0.94 & 0.93 \\ 1.75 & -1.54 \\ -1.46 & 1.58 \end{pmatrix} = (-5.14, 4.63)$$

Self-Attention

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

SoftMax(**Scaled Dot Product Similarities**)

	John	is	very	happy
John	0.98	0.01	0.00	0.01
is	0.37	0.22	0.14	0.28
very	0.03	0.19	0.64	0.14
happy	0.24	0.23	0.17	0.36

✖

V

-5.21	4.69
-0.94	0.93
1.75	-1.54
-1.46	1.58

=

Self-Attention values

-5.14	4.63
-2.25	2.12
0.61	-0.47
-1.69	1.65

$$(0.983, 0.005, 0, 0.012) \times \begin{pmatrix} -5.21 & 4.69 \\ -0.94 & 0.93 \\ 1.75 & -1.54 \\ -1.46 & 1.58 \end{pmatrix} = (-5.14, 4.63)$$

0.983 John + 0.005 is + 0 very + 0.012 happy = John_aggregated

Self-Attention

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

SoftMax(**Scaled Dot Product Similarities**)

	John	is	very	happy
John	0.98	0.01	0.00	0.01
is	0.37	0.22	0.14	0.28
very	0.03	0.19	0.64	0.14
happy	0.24	0.23	0.17	0.36

✖

V

-5.21	4.69
-0.94	0.93
1.75	-1.54
-1.46	1.58

=

Self-Attention values

-5.14	4.63
-2.25	2.12
0.61	-0.47
-1.69	1.65

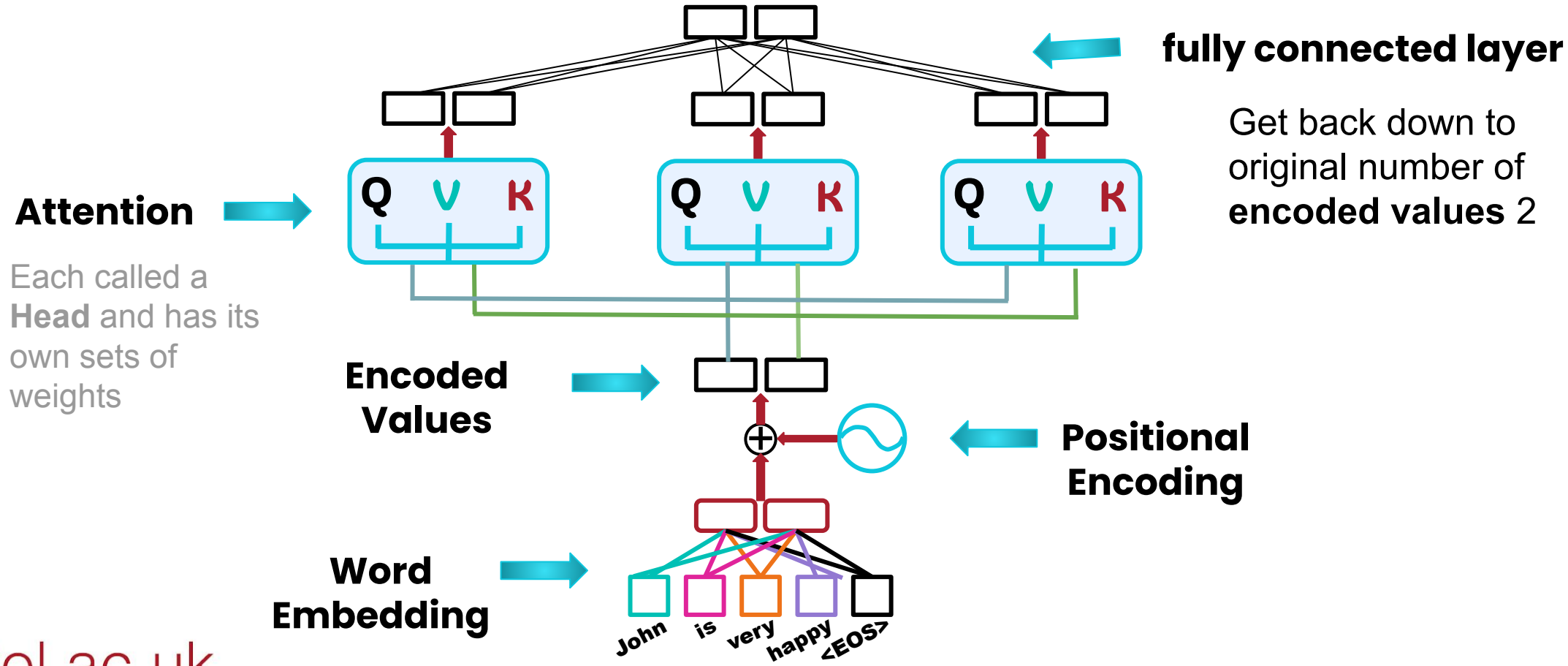
$$(0.983, 0.005, 0, 0.012) \times \begin{pmatrix} -5.21 & 4.69 \\ -0.94 & 0.93 \\ 1.75 & -1.54 \\ -1.46 & 1.58 \end{pmatrix} = (-5.14, 4.63)$$

Value/Embedding of 'John' is getting closer to 'happy'.

0.983 John + 0.005 is + 0 very + 0.012 happy = John_aggregated

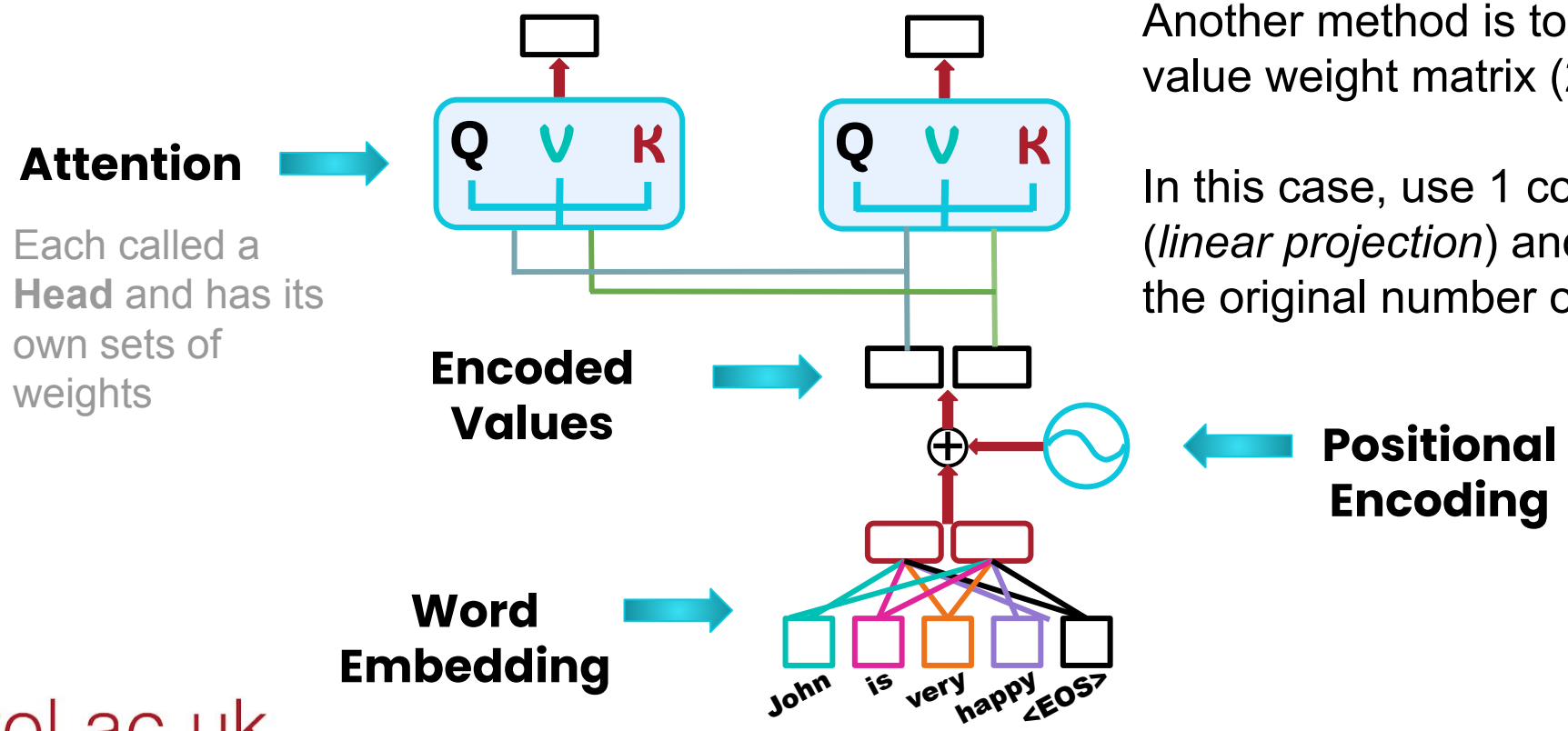
Multi-Head Attention

Apply Attention to the encoded Values multiple times simultaneously.



Multi-Head Attention

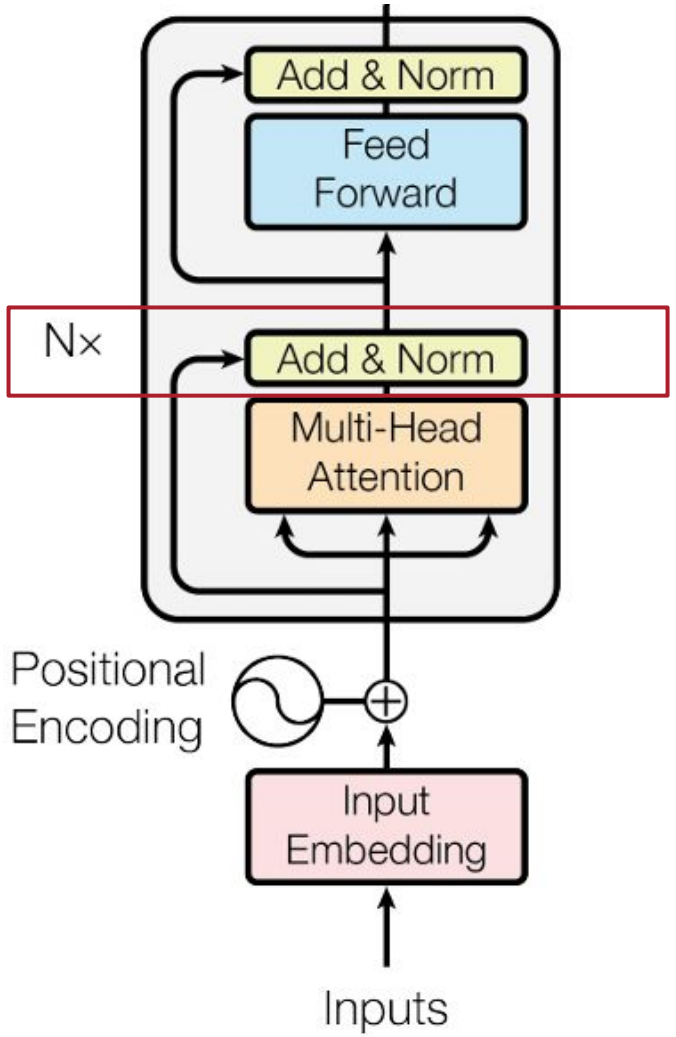
Apply Attention to the encoded Values multiple times simultaneously.

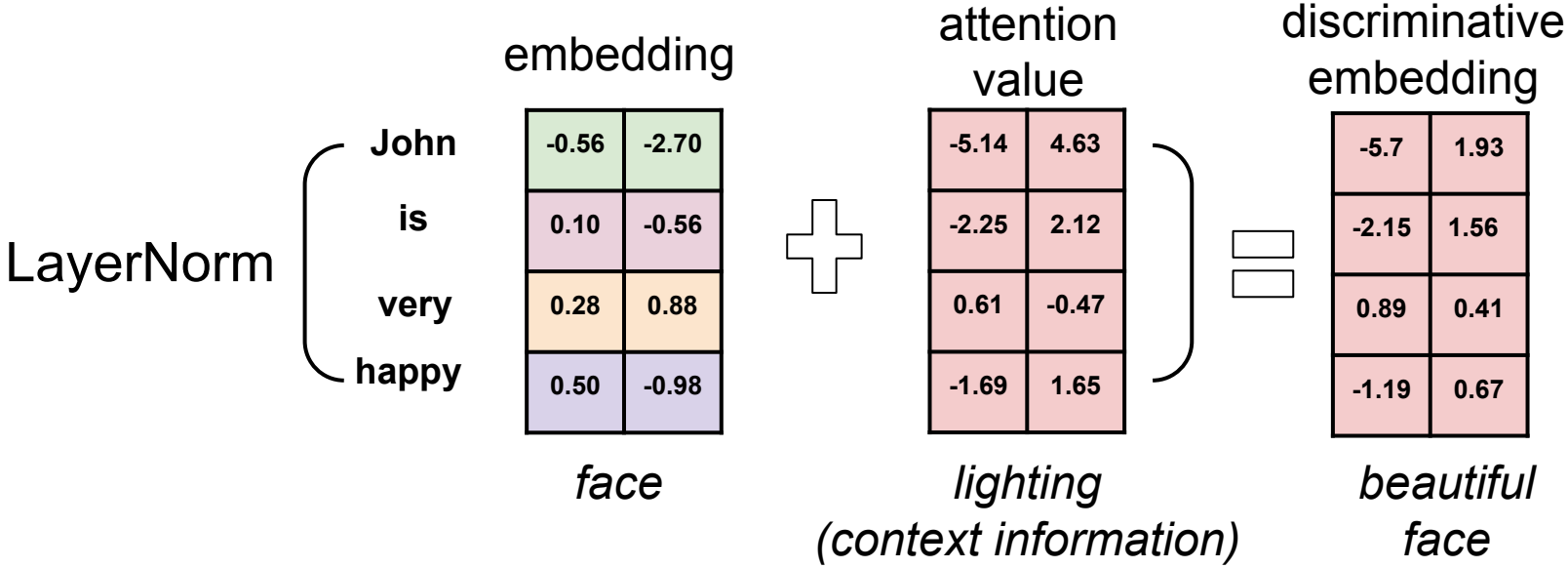


Another method is to modify the shape of value weight matrix (2->1).

In this case, use 1 column for weight matrix (*linear projection*) and 2 heads, then we get the original number of encoded values.

Self-Attention in Transformer

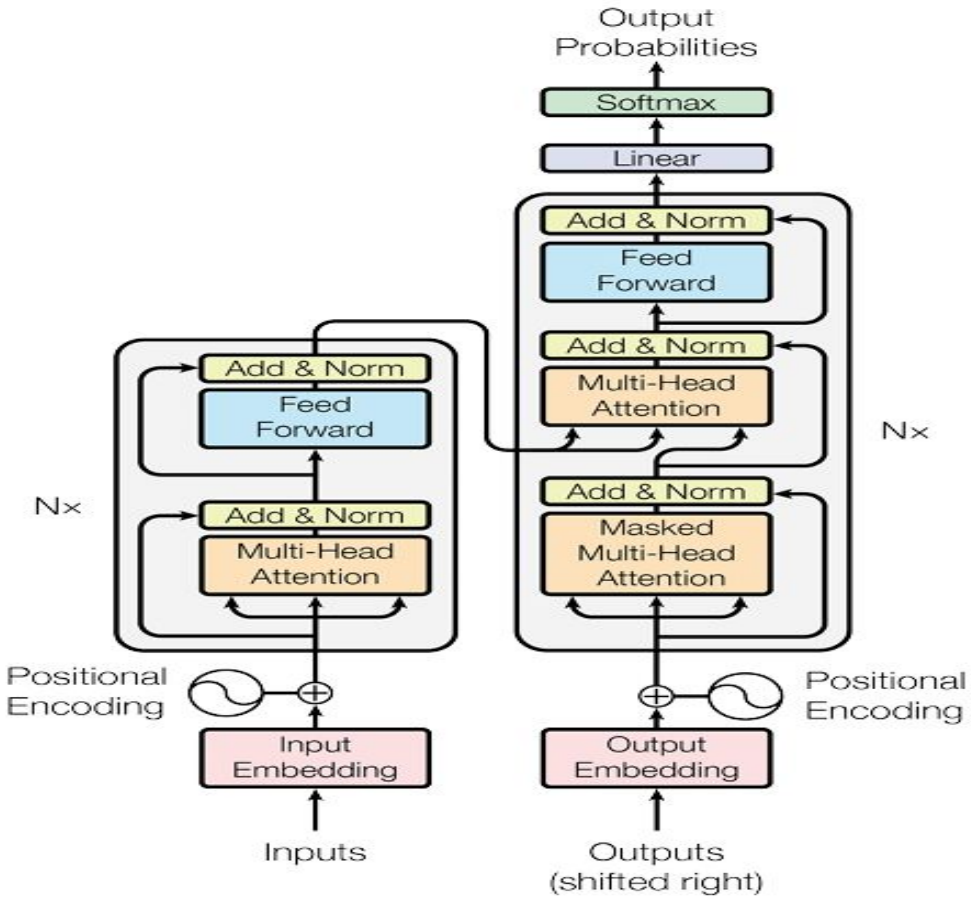




The addition operation can also be regarded as skip-connection.

Vaswani, Ashish, et al. "Attention is all you need." *NIPS* (2017).

Self-Attention in Transformer



Vaswani, Ashish, et al. "Attention is all you need." *NIPS* (2017).

Cross Attention

Build attention connections between two different sequences.

- **Self-Attention**: **Query, Key** and **Value** come from the same input sequence.
- **Cross-Attention**: one sequence provides the **Query**, another sequence provides the **Key** and **Value**.

- **Self-Attention** captures intra-sequence dependencies.
- **Cross-Attention** enables information fusion between two sequence.

Applications of Self-Attention

Example Application: “Non-Local Neural Networks “

The Non-Local Block introduces a self-attention mechanism that allows any two positions in a feature map to interact.

- In video understanding and image recognition, models must capture long-range dependencies.
- Traditional convolutions only model local neighborhoods → unable to capture global context.

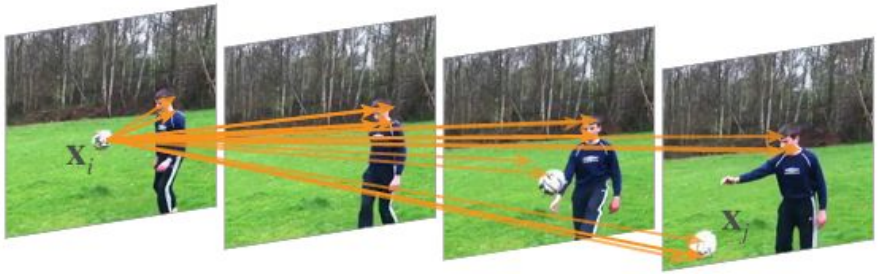


Figure 1. A spacetime ***non-local*** operation in our network trained for video classification in Kinetics. A position $\mathbf{x}_i$'s response is computed by the weighted average of the features of *all* positions $\mathbf{x}_j$ (only the highest weighted ones are shown here). In this example computed by our model, note how it relates the ball in the first frame to the ball in the last two frames. More examples are in Figure 3.

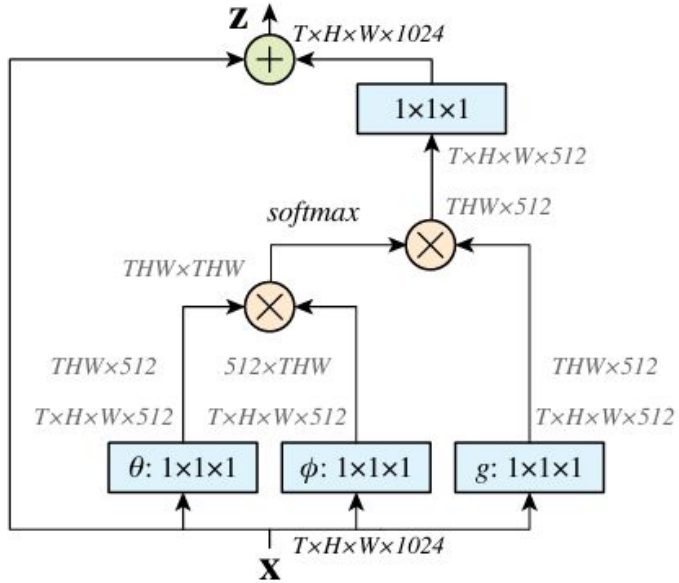


Figure 2. A spacetime **non-local block**. The feature maps are shown as the shape of their tensors, *e.g.*, $T \times H \times W \times 1024$ for 1024 channels (proper reshaping is performed when noted). “$\otimes$” denotes matrix multiplication, and “$\oplus$” denotes element-wise sum. The softmax operation is performed on each row. The blue boxes denote $1 \times 1 \times 1$ convolutions. Here we show the embedded Gaussian version, with a bottleneck of 512 channels. The vanilla Gaussian version can be done by removing θ and ϕ, and the dot-product version can be done by replacing softmax with scaling by $1/N$.

Wang, Xiaolong, et al. "Non-local neural networks." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018.

Applications of Self-Attention

Example Application: “End-to-End Object Detection with Transformers“

- DETR uses self-attention in its encoder to let every spatial position attend to all others.
- This enables global context modeling and captures relationships between objects.

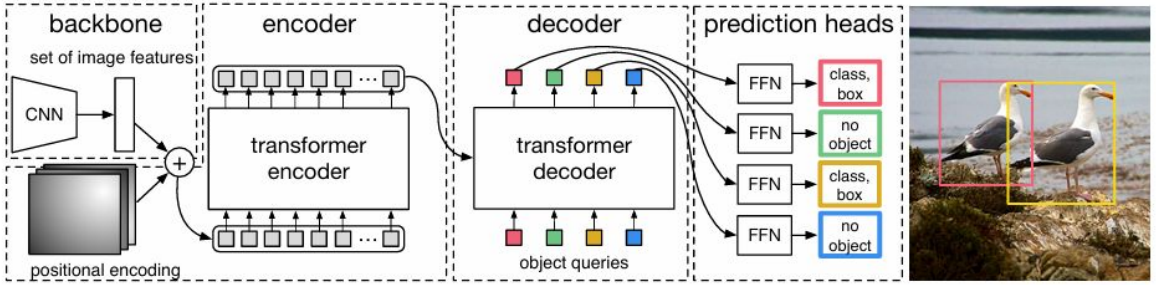


Fig. 2: DETR uses a conventional CNN backbone to learn a 2D representation of an input image. The model flattens it and supplements it with a positional encoding before passing it into a transformer encoder. A transformer decoder then takes as input a small fixed number of learned positional embeddings, which we call *object queries*, and additionally attends to the encoder output. We pass each output embedding of the decoder to a shared feed forward network (FFN) that predicts either a detection (class and bounding box) or a “`no object`” class.

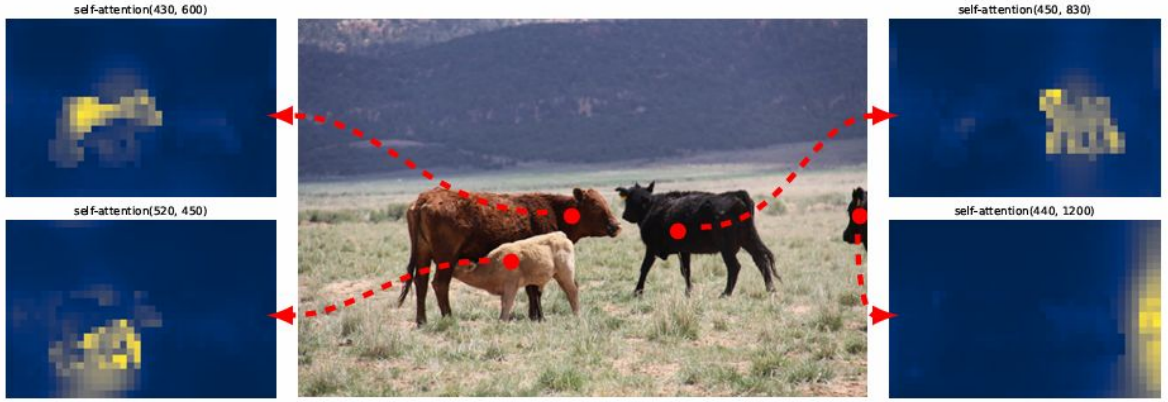


Fig. 3: Encoder self-attention for a set of reference points. The encoder is able to separate individual instances. Predictions are made with baseline DETR model on a validation set image.

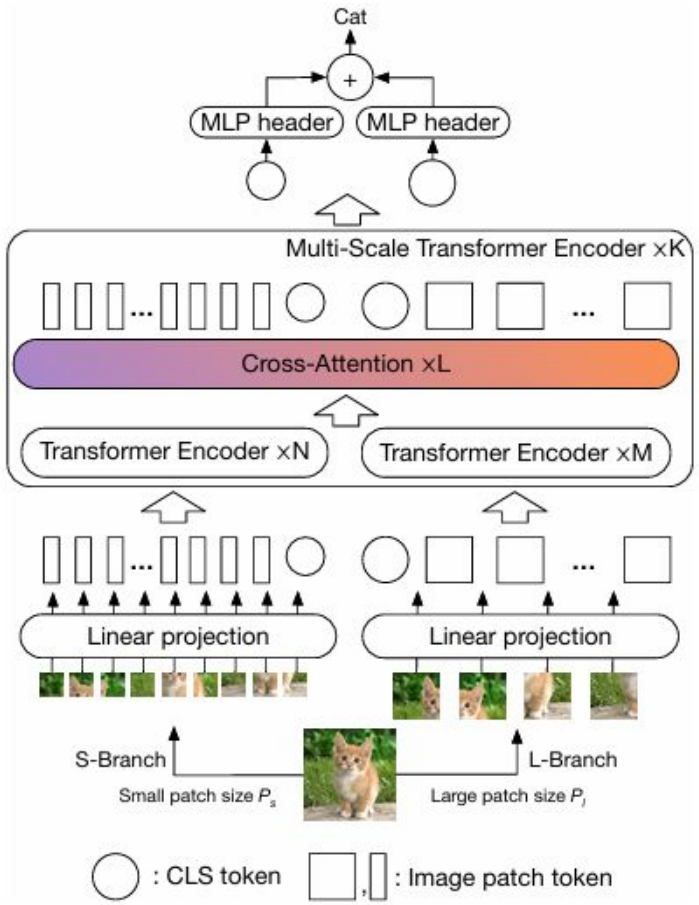
Carion, Nicolas, et al. "End-to-end object detection with transformers." *European conference on computer vision*. Cham: Springer International Publishing, 2020.

Applications of Cross-Attention

Example Application: “CrossViT: Cross-Attention Multi-Scale Vision Transformer for Image Classification“

CrossViT uses the cross-attention mechanism that lets small-patch and large-patch features interact.

- In image classification, models must capture both fine-grained details and coarse global structure.
- Traditional ViTs using a single patch size → unable to integrate multi-scale information effectively.



Chen, Chun-Fu Richard, Quanfu Fan, and Rameswar Panda. "Crossvit: Cross-attention multi-scale vision transformer for image classification." *Proceedings of the IEEE/CVF international conference on computer vision*. 2021.

Applications of Cross-Attention

Example Application: “BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation“

BLIP uses the cross-attention mechanism that allows textual tokens and visual tokens to attend to each other.

- In vision-language tasks, models must capture relationships between text and image regions.
- Traditional single-modality encoders only model one modality → unable to integrate multi-modal information effectively

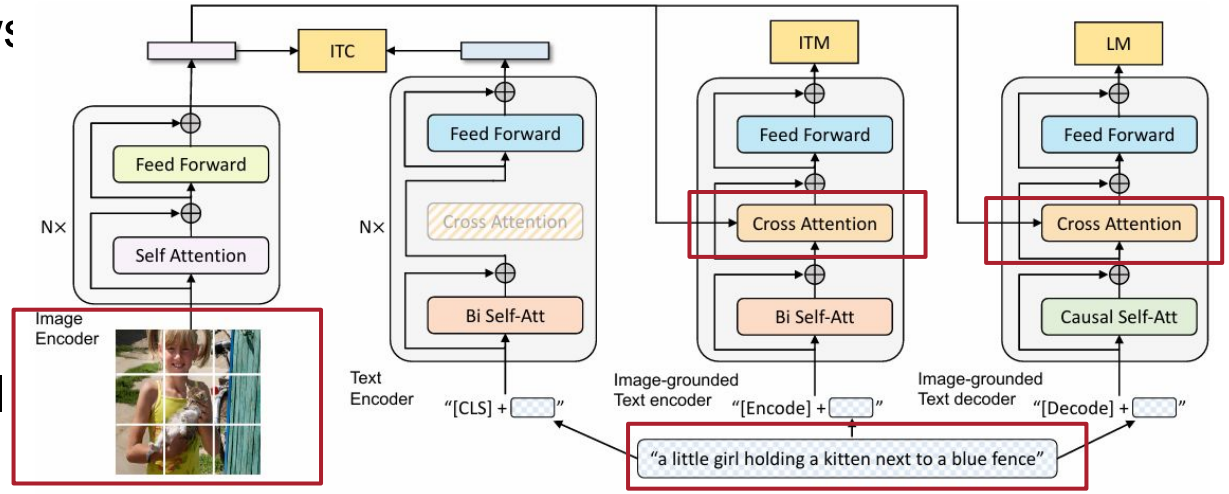


Figure 2. Pre-training model architecture and objectives of BLIP (same parameters have the same color). We propose multimodal mixture of encoder-decoder, a unified vision-language model which can operate in one of the three functionalities: (1) Unimodal encoder is trained with an image-text contrastive (ITC) loss to align the vision and language representations. (2) Image-grounded text encoder uses additional cross-attention layers to model vision-language interactions, and is trained with a image-text matching (ITM) loss to distinguish between positive and negative image-text pairs. (3) Image-grounded text decoder replaces the bi-directional self-attention layers with causal self-attention layers, and shares the same cross-attention layers and feed forward networks as the encoder. The decoder is trained with a language modeling (LM) loss to generate captions given images.

Li, Junnan, et al. "Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation." *International conference on machine learning*. PMLR, 2022.